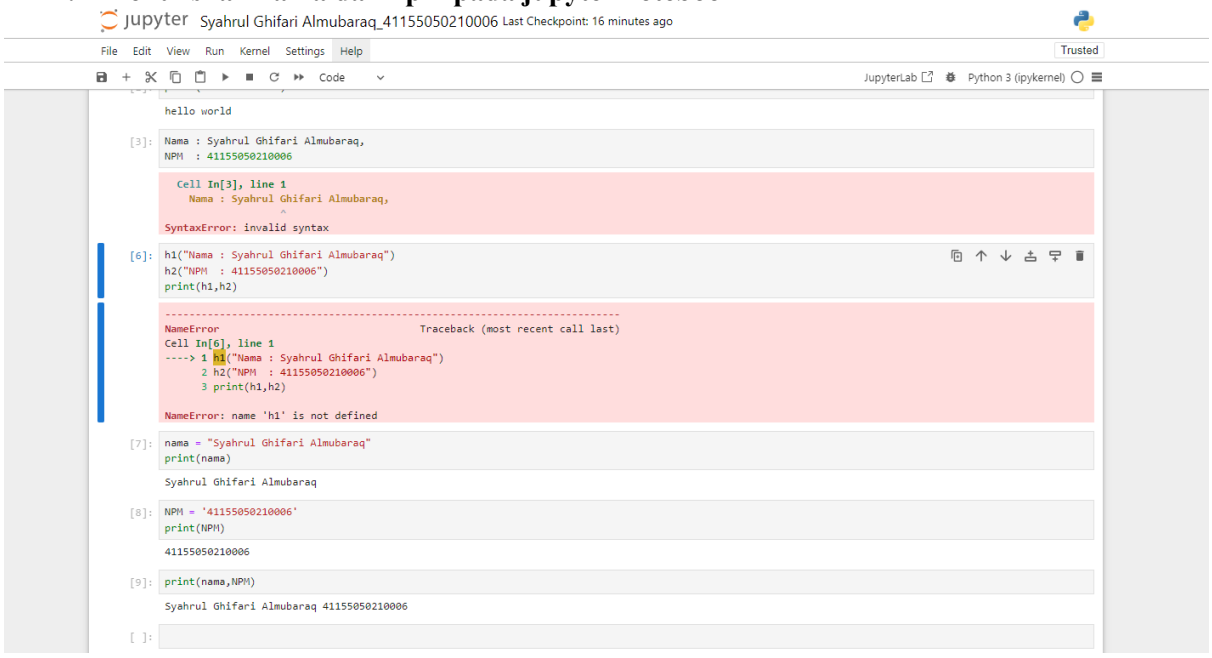
Nama : Syahrul Ghifari Almubaraq

NPM : 41155050210006

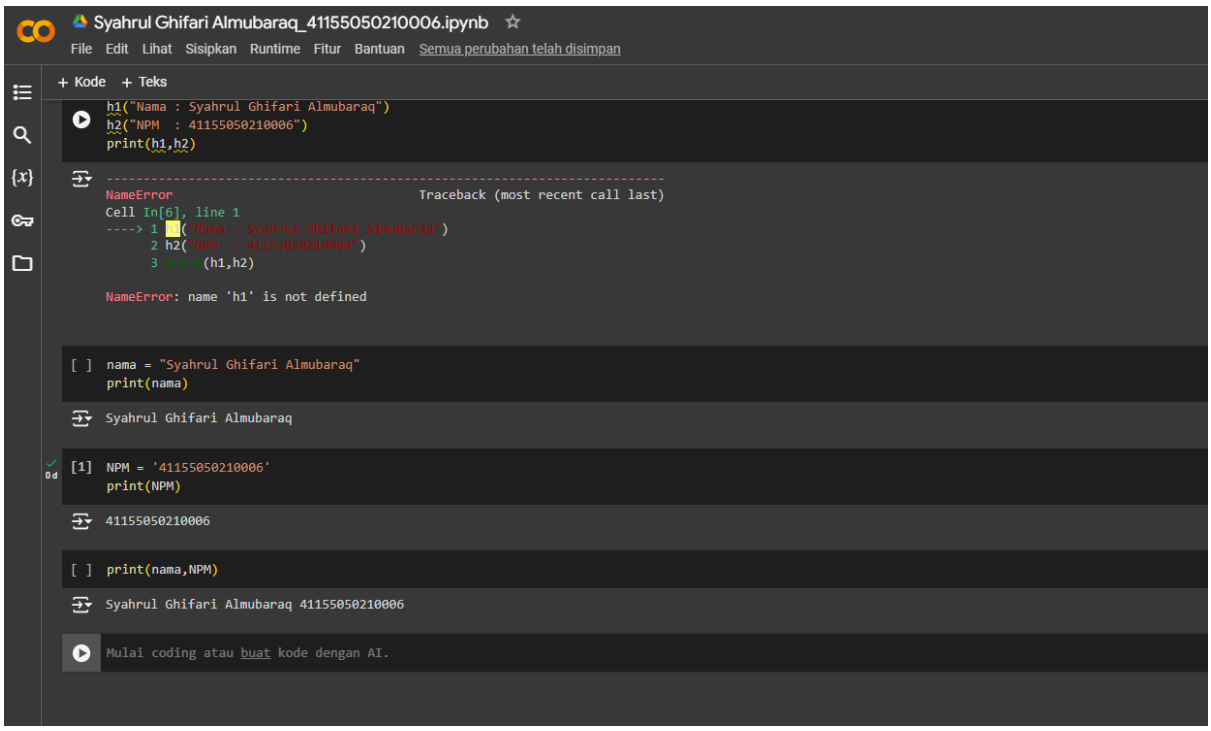
Kelas : INF A1

Jawaban!

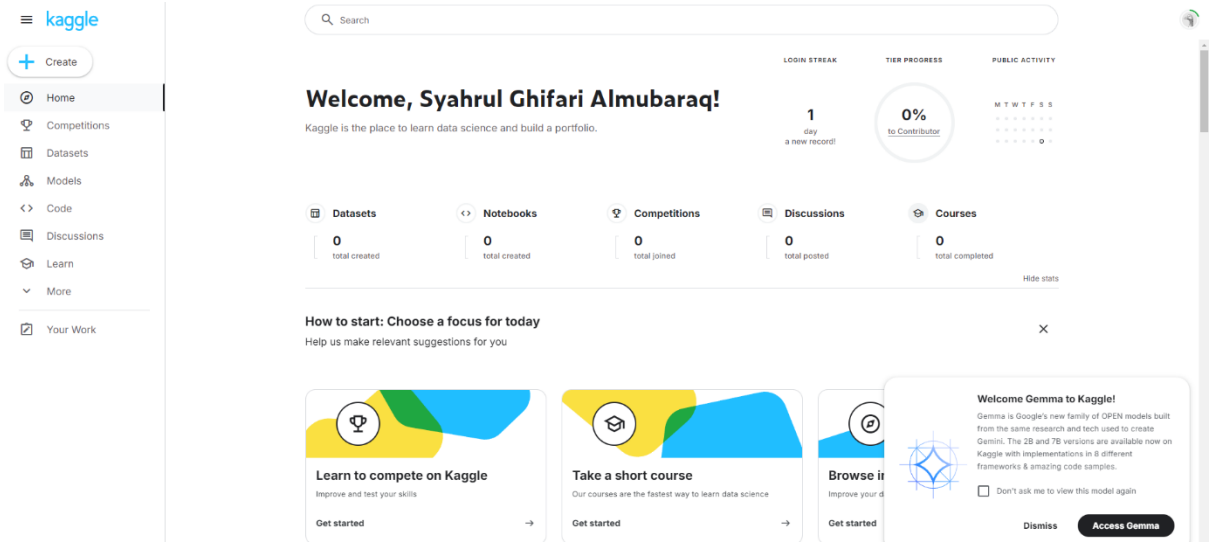
1. **Menuliskan nama dan npm pada jupyter notebook**



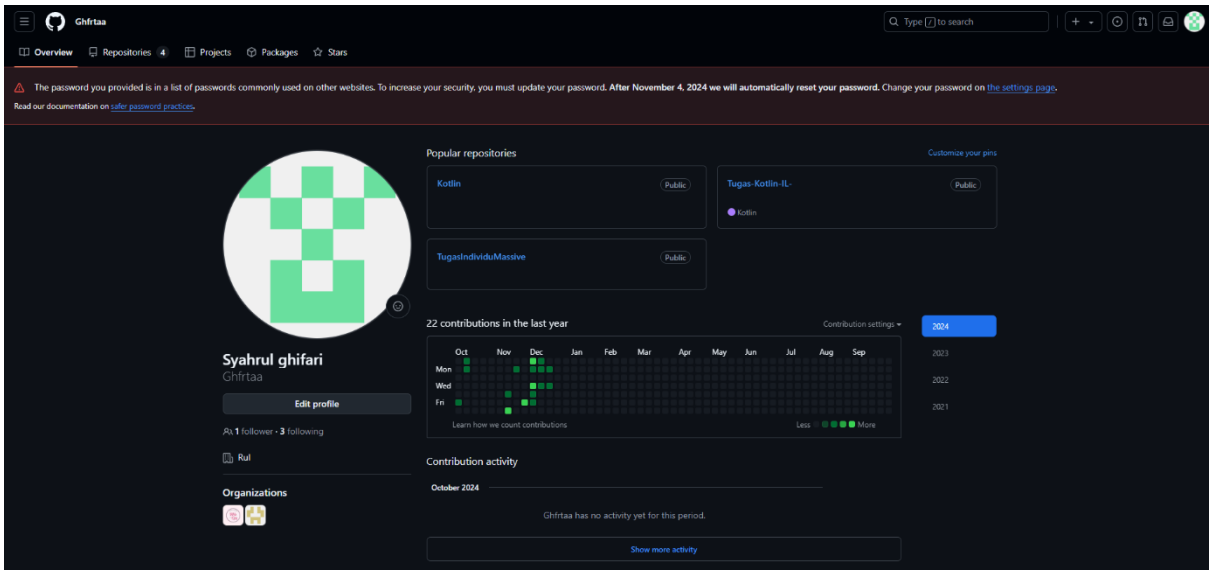
2. **Menuliskan Nama dan NPM pada Google Colabs, dan dikarenakan dari google colabs itu sama menggunakan notebook maka dari itu saya mengambil datanya pada jupyter notebook.**



3. **Membuat Akun kaggle saya disini registrasinya menggunakan akun gmail.**



4. **Akun Github.**



5. **Tugas 5 Di pertemuan 1**
   - **Load Sample Dataset**

```
[15]: from sklearn.datasets import load_iris

      iris = load_iris()
      iris
```

```
[15]: {'data': array([[5.1, 3.5, 1.4, 0.2],
                      [4.9, 3. , 1.4, 0.2],
                      [4.7, 3.2, 1.3, 0.2],
                      [4.6, 3.1, 1.5, 0.2],
                      [5. , 3.6, 1.4, 0.2],
                      [5.4, 3.9, 1.7, 0.4],
                      [4.6, 3.4, 1.4, 0.3],
                      [5. , 3.4, 1.5, 0.2],
                      [4.4, 2.9, 1.4, 0.2],
                      [4.9, 3.1, 1.5, 0.1],
                      [5.4, 3.7, 1.5, 0.2],
                      [4.8, 3.4, 1.6, 0.2],
                      [4.8, 3. , 1.4, 0.1],
                      [4.3, 3. , 1.1, 0.1],
                      [5.8, 4. , 1.2, 0.2],
                      [5.7, 4.4, 1.5, 0.4],
                      [5.4, 3.9, 1.3, 0.4],
                      [5.1, 3.5, 1.4, 0.3],
```

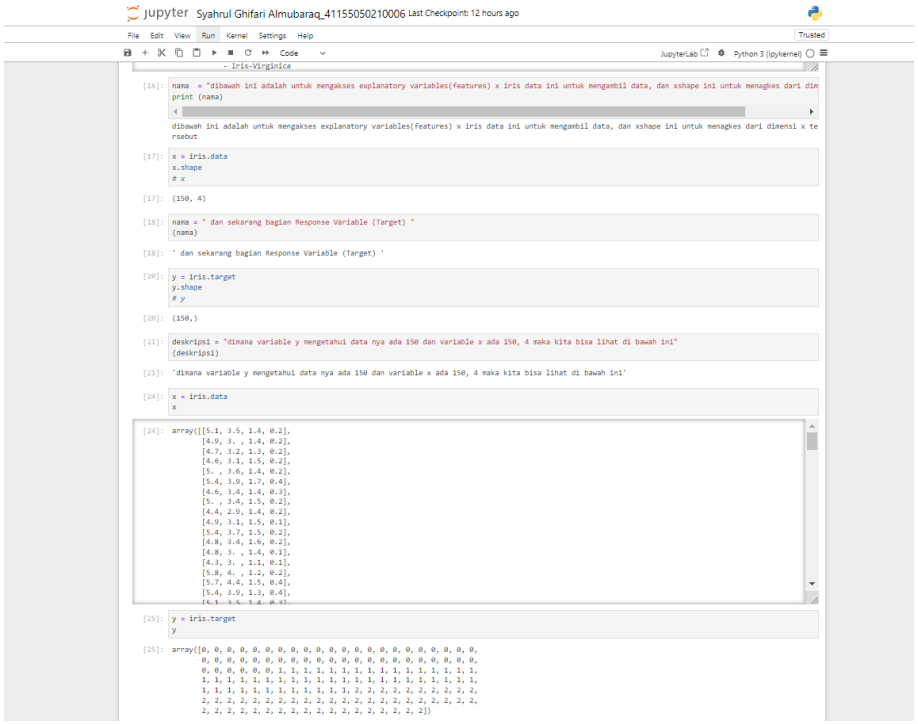
```
[16]: iris.keys()
```

```
[16]: dict_keys(['data', 'target', 'frame', 'target_names', 'DESCR', 'feature_names', 'filename', 'data_module'])
```

- **Metadata | Deskripsi dari sampl dataset**

```
[17]: print(iris.DESCR)

.. _iris_dataset:

Iris plants dataset
--------------------

**Data Set Characteristics:**

    :Number of Instances: 150 (50 in each of three classes)
    :Number of Attributes: 4 numeric, predictive attributes and the class
    :Attribute Information:
        - sepal length in cm
        - sepal width in cm
        - petal length in cm
        - petal width in cm
        - class:
                - Iris-Setosa
                - Iris-Versicolour
                - Iris-Virginica

    :Summary Statistics:

    ============== ==== ==== ======= ===== ====================
                    Min  Max   Mean    SD   Class Correlation
    ============== ==== ==== ======= ===== ====================
    sepal length:   4.3  7.9   5.84   0.83    0.7826
    sepal width:    2.0  4.4   3.05   0.43   -0.4194
    petal length:   1.0  6.9   3.76   1.76    0.9490  (high!)
    petal width:    0.1  2.5   1.20   0.76    0.9565  (high!)
    ============== ==== ==== ======= ===== ====================

    :Missing Attribute Values: None
    :Class Distribution: 33.3% for each of 3 classes.
    :Creator: R.A. Fisher
    :Donor: Michael Marshall (MARSHALL%PLU@io.arc.nasa.gov)
    :Date: July, 1988

    The famous Iris database, first used by Sir R.A. Fisher. The dataset is taken
    from Fisher's paper. Note that it's the same as in R, but not as in the UCI
    Machine Learning Repository, which has two wrong data points.

    This is perhaps the best known database to be found in the
    pattern recognition literature.  Fisher's paper is a classic in the field and
    is referenced frequently to this day.  (See Duda & Hart, for example.)  The
    data set contains 3 classes of 50 instances each, where each class refers to a
```

- **Explanatory & Response Variables (feature & target)**



- **Feature & Target Names**
  Untuk memahami makna kita bisa pada gambar di bawah ini.

```
       [6.4, 2.8, 5.6, 2.2],
       [6.3, 2.8, 5.1, 1.5],
       [6.1, 2.6, 5.6, 1.4],
       [7.7, 3. , 6.1, 2.3],
       [6.3, 3.4, 5.6, 2.4],
       [6.4, 3.1, 5.5, 1.8],
       [6. , 3. , 4.8, 1.8],
       [6.9, 3.1, 5.4, 2.1],
       [6.7, 3.1, 5.6, 2.4],
       [6.9, 3.1, 5.1, 2.3],
       [5.8, 2.7, 5.1, 1.9],
       [6.8, 3.2, 5.9, 2.3],
       [6.7, 3.3, 5.7, 2.5],
       [6.7, 3. , 5.2, 2.3],
       [6.3, 2.5, 5. , 1.9],
       [6.5, 3. , 5.2, 2. ],
       [6.2, 3.4, 5.4, 2.3],
       [5.9, 3. , 5.1, 1.8]])
```

```
[25]: y = iris.target
      y
```

```
[25]: array([0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
       0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
       0, 0, 0, 0, 0, 0, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1,
       1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1,
       1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2,
       2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2,
       2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2])
```

```
[28]: feature_names = iris.feature_names
      feature_names
```

```
[28]: ['sepal length (cm)',
       'sepal width (cm)',
       'petal length (cm)',
       'petal width (cm)']
```

```
[29]: target = iris.target_names
      target
```

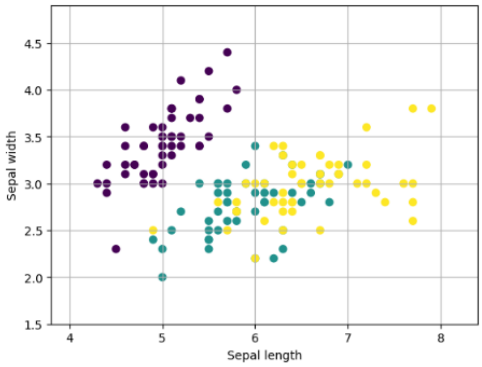
```
[29]: array(['setosa', 'versicolor', 'virginica'], dtype='<U10')
```

```
[ ]:
```

- **Visualisasi Data**

  Visualisasi Sepal length & Width

```
[32]: import matplotlib.pyplot as plt

      x = x[:, :2]

      x_min, x_max = x[:, 0].min() - 0.5, x[:, 0].max() + 0.5
      y_min, y_max = x[:, 1].min() - 0.5, x[:, 1].max() + 0.5

      plt.scatter(x[:, 0], x[:, 1], c=y)
      plt.xlabel('Sepal length')
      plt.ylabel('Sepal width')

      plt.xlim(x_min, x_max)
      plt.ylim(y_min, y_max)
      plt.grid(True)
      plt.show()
```




```
[ ]:
```

- **Training & Testing Dataset**

```
[33]: from sklearn.model_selection import train_test_split

      x_train, x_test, y_train, y_test = train_test_split(x,
                                                          y,
                                                          test_size=0.3,
                                                          random_state=1)

      print(f'x train: {x_train.shape}')
      print(f'x test: {x_test.shape}')
      print(f'y train: {y_train.shape}')
      print(f'y test: {y_test.shape}')
```

```
x train: (105, 2)
x test: (45, 2)
y train: (105,)
y test: (45,)
```

```
[ ]:
```

- **Load Iris Dataset sebagai Pandas DataFrame(DF)**

```
[34]: iris = load_iris(as_frame=True)

      iris_features_df = iris.data
      iris_features_df
```

[34]:

	sepal length (cm)	sepal width (cm)	petal length (cm)	petal width (cm)
0	5.1	3.5	1.4	0.2
1	4.9	3.0	1.4	0.2
2	4.7	3.2	1.3	0.2
3	4.6	3.1	1.5	0.2
4	5.0	3.6	1.4	0.2
...	...	...	...	...
145	6.7	3.0	5.2	2.3
146	6.3	2.5	5.0	1.9
147	6.5	3.0	5.2	2.0
148	6.2	3.4	5.4	2.3
149	5.9	3.0	5.1	1.8

150 rows × 4 columns

6. **Tugas 6 Di pertemuan 1**

- **Persiapan Dataset | Loading & Splitting dataset**

```
[36]: from sklearn.datasets import load_iris

      iris = load_iris()

      x = iris.data
      y = iris.target
```

```
[37]: from sklearn.model_selection import train_test_split
      x_train, x_test, y_train, y_test = train_test_split(x,
                                                          y,
                                                          test_size=0.4,
                                                          random_state=1)
```

- **Training Model Machine Learning**

  - Pada Scikit Learn, model machine learning dibentuk dari class yang dikenal dengan istilah **estimator**.
  - Setiap estimator akan mengimplementasikan dua method utama, yaitu `fit()` dan `predict()`.
  - Method `fit()` digunakan untuk melakukan training model.
  - Method `predict()` digunakan untuk melakukan estimasi/prediksi dengan memanfaatkan trained model.

```
[38]: from sklearn.neighbors import KNeighborsClassifier

      model = KNeighborsClassifier(n_neighbors=3)
      model.fit(x_train, y_train)
```

```
[38]: KNeighborsClassifier
      KNeighborsClassifier(n_neighbors=3)
```

- **Evaluasi model Machine Learning**
  Yang dimaksud disini adalah nilai akurasi dari model kita adalah sekitar 98%

```
[39]: from sklearn.metrics import accuracy_score

      y_pred = model.predict(x_test)
      acc = accuracy_score(y_test, y_pred)
      print(f'accuracy: {acc}')

      accuracy: 0.9833333333333333
```

- **Pemanfaatan trained model machine learning**

```
[43]: data_baru = [[5, 5, 3, 2],
                   [2, 4, 3, 5]]
      preds = model.predict(data_baru)
      preds
```

```
[43]: array([1, 2])
```

```
[44]: pred_species = [iris.target_names[p] for p in preds]
      print(f'Hasil prediksi: {pred_species}')
```

```
Hasil prediksi: [np.str_('versicolor'), np.str_('virginica')]
```

- **Deploy model Machine Learning | Dumping dan Loading model Machine Learning**

```
[45]: import joblib

      joblib.dump(model, 'iris_classifier_knn.joblib')
```

```
[45]: ['iris_classifier_knn.joblib']
```

```
[46]: production_model = joblib.load('iris_classifier_knn.joblib')
```

7. **Tugas 7 Di pertemuan 1**
   - **Persiapan sample dataset.**

```
[48]: import numpy as np
      from sklearn import preprocessing

      sample_data = np.array([[2.1, -1.9, 5.5],
                              [-1.5, 2.4, 3.5],
                              [0.5, -7.9, 5.6],
                              [5.9, 2.3, -5.8]])
      sample_data
```

```
[48]: array([[ 2.1, -1.9,  5.5],
             [-1.5,  2.4,  3.5],
             [ 0.5, -7.9,  5.6],
             [ 5.9,  2.3, -5.8]])
```

```
[49]: sample_data.shape
```

```
[49]: (4, 3)
```

   - **Teknik data preprocessing 1:binarisation.**

```
[50]: sample_data
```

```
[50]: array([[ 2.1, -1.9,  5.5],
             [-1.5,  2.4,  3.5],
             [ 0.5, -7.9,  5.6],
             [ 5.9,  2.3, -5.8]])
```

```
[52]: preprocessor = preprocessing.Binarizer(threshold=0.5)
      binarised_data = preprocessor.transform(sample_data)
      binarised_data
```

```
[52]: array([[1., 0., 1.],
             [0., 1., 1.],
             [0., 0., 1.],
             [1., 1., 0.]])
```

   - **Teknik data preprocessing 2:scaling.**

```
[55]: preprocessor = preprocessing.MinMaxScaler(feature_range=(0,1))
      preprocessor.fit(sample_data)
      scaled_data = preprocessor.transform(sample_data)
      scaled_data
```

```
[55]: array([[0.48648649, 0.58252427, 0.99122807],
             [0.        , 1.        , 0.81578947],
             [0.27027027, 0.        , 1.        ],
             [1.        , 0.99029126, 0.        ]])
```

```
[56]: scaled_data = preprocessor.fit_transform(sample_data)
      scaled_data
```

```
[56]: array([[0.48648649, 0.58252427, 0.99122807],
             [0.        , 1.        , 0.81578947],
             [0.27027027, 0.        , 1.        ],
             [1.        , 0.99029126, 0.        ]])
```

- **Teknik data preprocessing 3: normalisation.**
  Normalisasi L1 Least Absolute Deviations

```
[57]: sample_data
```

```
[57]: array([[ 2.1, -1.9,  5.5],
             [-1.5,  2.4,  3.5],
             [ 0.5, -7.9,  5.6],
             [ 5.9,  2.3, -5.8]])
```

```
[58]: l1_normalised_data = preprocessing.normalize(sample_data, norm='l1')
      l1_normalised_data
```

```
[58]: array([[ 0.22105263, -0.2       ,  0.57894737],
             [-0.2027027 ,  0.32432432,  0.47297297],
             [ 0.03571429, -0.56428571,  0.4       ],
             [ 0.42142857,  0.16428571, -0.41428571]])
```

```
[ ]:
```

Normalisasi 2 Least Squares

```
[59]: sample_data
```

```
[59]: array([[ 2.1, -1.9,  5.5],
             [-1.5,  2.4,  3.5],
             [ 0.5, -7.9,  5.6],
             [ 5.9,  2.3, -5.8]])
```

```
[60]: l2_normalised_data = preprocessing.normalize(sample_data, norm='l2')
      l2_normalised_data
```

```
[60]: array([[ 0.33946114, -0.30713151,  0.88906489],
             [-0.33325106,  0.53320169,  0.7775858 ],
             [ 0.05156558, -0.81473612,  0.57753446],
             [ 0.68706914,  0.26784051, -0.6754239 ]])
```